

bok25

**基
礎
班**

、考研数据結構



并查集



并查集

定义

并查集用于处理一些不交集的合并及查询问题，支持以下操作：

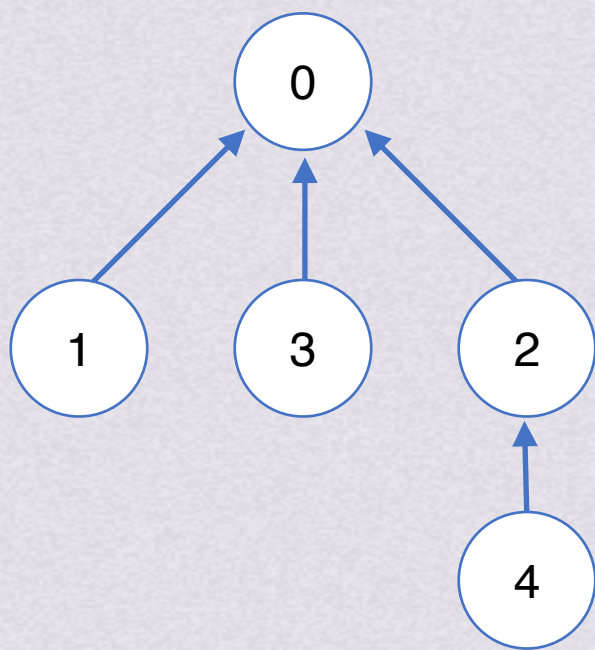
- **Initial (S)** ： 将集合S中的每个元素都初始化为单元素子集合。
- **Find (S, x)** ： 查询某个元素属于哪个集合，通常是返回集合内的一个“代表元素”。这个操作是为了判断两个元素是否在同一个集合之中。
- **Union (S, root1, root2)** ： 将两个集合合并为一个。



并查集

存储

双亲表示法存储!



存储数据	-1	0	0	0	2
数组下标	0	1	2	3	4



并查集

初始化



并查集

初始化

```
#define MAXSIZE 10  
void Initial(int S[])  
{  
    for(int i=0;i<MAXSIZE;i++)  
        s[i]=-1;  
}
```



并查集

查找



并查集

查找

查找元素x并返回x所在集合的根。时间复杂度为 $O(h)$ ，h为集合树的高度。

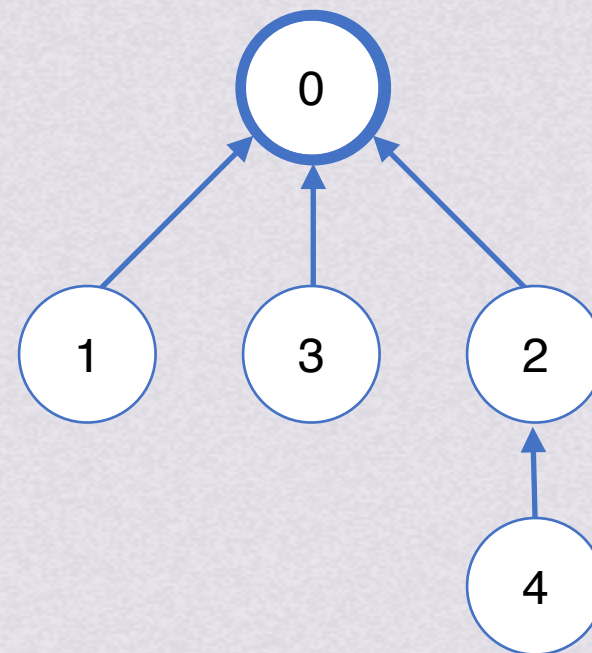
```
int Find(int S[], int x)
{
    while(S[x] >= 0)
        x = S[x];
    return x;
}
```



并查集查找

查找元素x并返回x所在集合的根。时间复杂度为 $O(h)$ ，h为集合树的高度。

```
int Find(int S[], int x) 传入S[],4
{
    while(S[x] >= 0) S[x]=S[0]=0
    x=S[x]; x=S[2] (x=0)
    return x; 返回x=0
}
```



存储数据

-1	0	0	0	2
----	---	---	---	---

数组下标

0 1 2 3 4



并查集

查找的优化
路径压缩



并查集

查找的优化
路径压缩

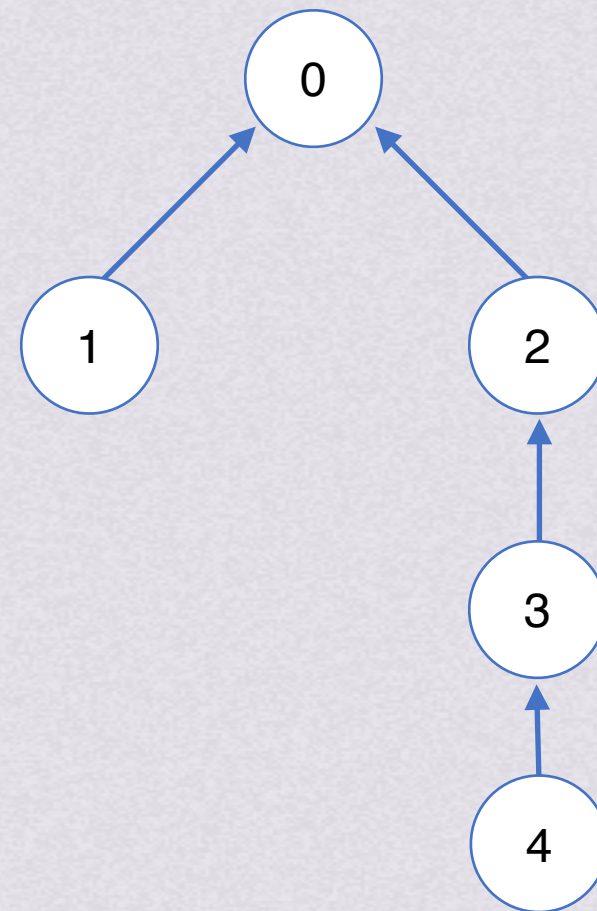
```
int Find(int S[],int x)
{
    int root=x;
    while(S[root]>=0)
        root=S[root];
    while(x!=root)
    {
        int t=S[x];
        S[x]=root;
        x=t;
    }
    return root;
}
```



并查集

查找的优化
路径压缩

```
int Find(int S[],int x)
{
    int root=x;
    while(S[root]>=0)
        root=S[root];
    while(x!=root)
    {
        int t=S[x];
        S[x]=root;
        x=t;
    }
    return root;
}
```



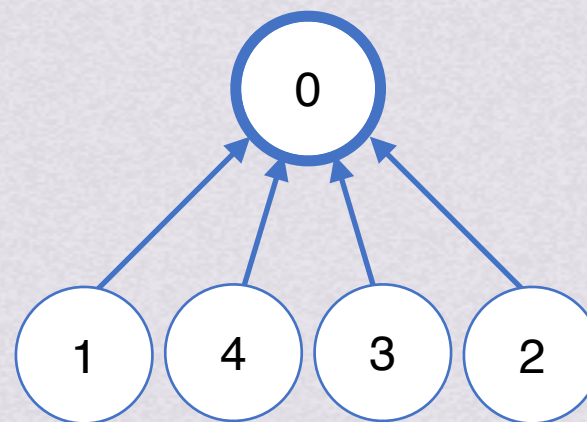
存储数据	-1	0	0	2	3
数组下标	0	1	2	3	4



并查集

查找的优化
路径压缩

```
int Find(int S[],int x) 传入S[], 4
{
    int root=x; root = 4
    while(S[root]>=0) S[0] = -1 不进入循环
        root=S[root]; root = S[2] = 0
    while(x!=root) x=0 root=0 x=root
        不进入循环
    {
        int t=S[x]; t=S[2]=0
        S[x]=root; S[2]=0
        x=t; x=0
    }
    return root; return 0
}
```



存储数据	-1	0	0	0	0
数组下标	0	1	2	3	4



并查集

合并



并查集

合并

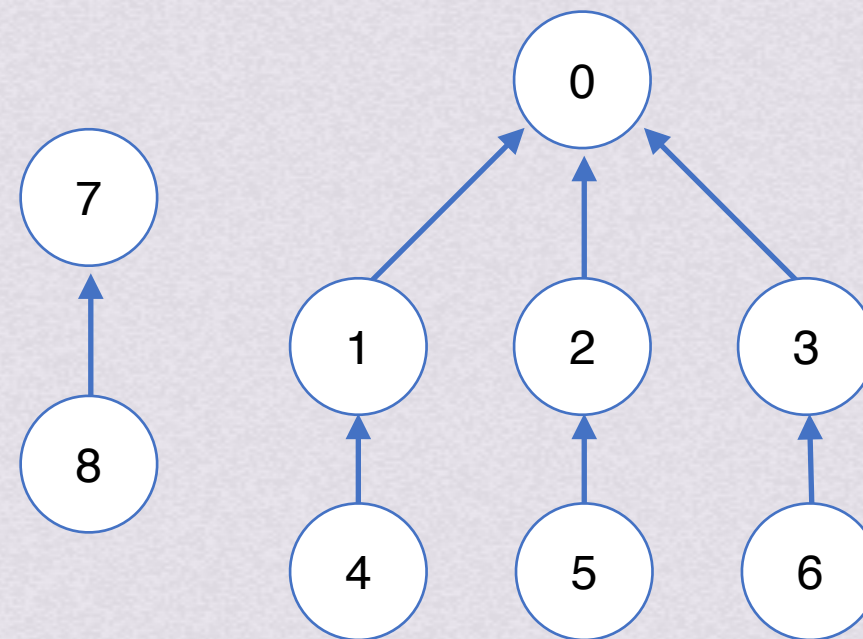
```
void Union(int S[],int root1,int root2)
{
    if(root1==root2)
        return;
    S[root2]=root1;
}
```



并查集

合并

```
void Union(int S[],int root1,int root2)
{
    if(root1==root2)
        return;
    S[root2]=root1;
}
```



存储数据

-1	0	0	0	1	2	3	-1	7
----	---	---	---	---	---	---	----	---

数组下标

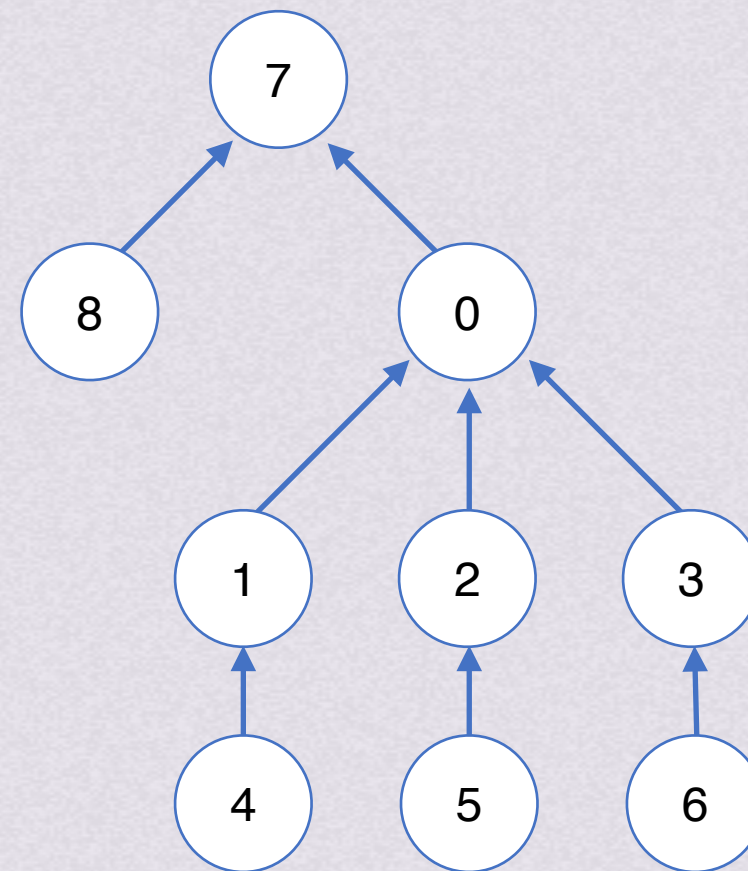
0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---



并查集合并

```
void Union(int S[],int root1,int root2)
{
    if(root1==root2) 7≠0 跳过if
        return;
    S[root2]=root1; S[0] = 7
}
```

传入S[], 7, 0



存储数据

数组下标

7	0	0	0	1	2	3	-1	7
0	1	2	3	4	5	6	7	8

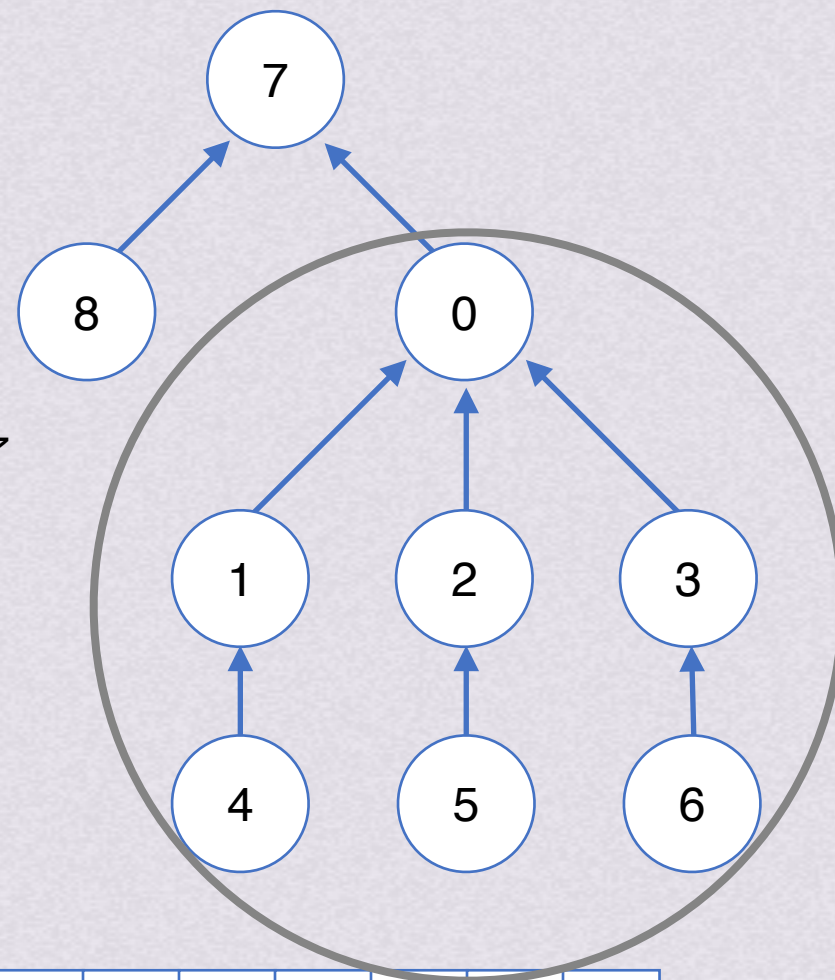


并查集

合并的优化

普通合并方式，0,1,2,3,4,5,6在日后寻找根的过程中，搜索长度都变长了

通过控制必须**小集合合并到大集合之下**，
可以只让本例中的7,8寻找根的长度变长



存储数据

数组下标

7	0	0	0	1	2	3	-1	7
0	1	2	3	4	5	6	7	8

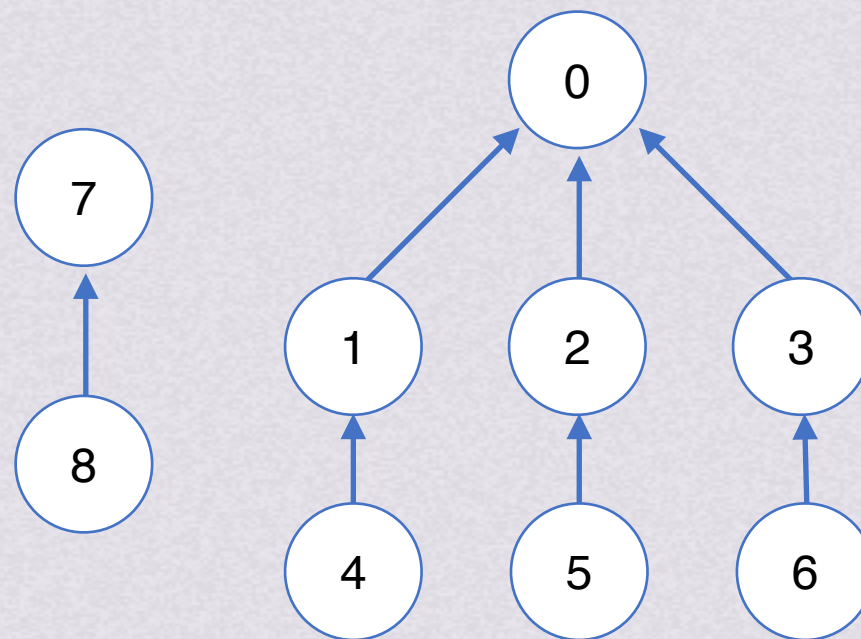


并查集合并的优化

普通合并方式，0,1,2,3,4,5,6在日后寻找根的过程中，搜索长度都变长了

通过控制必须**小集合合并到大集合之下**，
可以只让本例中的7,8寻找根的长度变长

```
void Union(int S[],int root1,int root2)
{
    if(root1==root2) return;
    if(S[root2]>S[root1])
    {
        S[root1]=S[root1]+S[root2];
        S[root2]=root1;
    }
    else
    {
        S[root2]=S[root1]+S[root2];
        S[root1]=root2;
    }
}
```



存储数据

-7

0

0

0

1

2

3

-2

7

数组下标

0

1

2

3

4

5

6

7

8

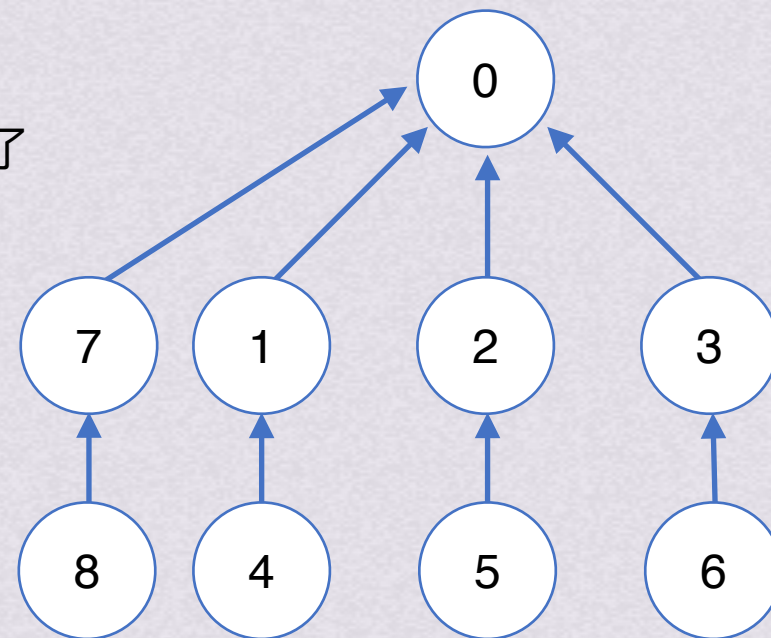
根节点需要标记本集合元素的个数



并查集合并的优化

普通合并方式，0,1,2,3,4,5,6在日后寻找根的过程中，搜索长度都变长了
通过控制必须**小集合合并到大集合之下**，
可以只让本例中的7,8寻找根的长度变长

```
void Union(int S[],int root1,int root2)
{
    if(root1==root2) return;
    if(S[root2]>S[root1])
    {
        S[root1]=S[root1]+S[root2];
        S[root2]=root1;
    }
    else
    {
        S[root2]=S[root1]+S[root2];
        S[root1]=root2;
    }
}
```



存储数据

-9	0	0	0	1	2	3	0	7
----	---	---	---	---	---	---	---	---

数组下标

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

改进后的合并操作可以使最终的集合树高度不超过 $\lfloor \log_2 n \rfloor + 1$